INF-122 Obligatorisk oppgave, H-2026

- Les hele oppgaven før du begynner å jobbe med en løsning.
- Det er lov å arbeide alene eller i grupper på to personer. (Ta kontakt med en gruppeleder eller foreleser hvis dere absolutt vil ha én gruppe på tre personer.)

  Kode fra forelesninger, boken og oppgaver kan gjenbrukes men duplisering av kode fra andre kilder er ikke tillatt. Spesielt, skal man ikke bruke KI til å programmere.
- Man kan stille spørsmål og jobbe med oppgaven på grupper fom. 22.10 tom. 6.11.
- Innleveringsfrist er **fredag, 6 November 2026, kl. 16:00**. Senere innleveringer blir ikke vurdert. Det blir ikke tid til andre forsøk, så første innleveringen må bli godkjent for å kunne gå opp til eksamen.
- Løsning lastes opp som en enkel kjørbar .hs fil under Oppgaver/Oblig på kursets webside. Koden skal
  – starte med en kommentar med navn på dem som innleverer denne løsningen, og
  – inneholde GHCi-kjørbar Haskell kode (ikke endre typer til funksjoner spesifisert i oppgaveteksten)
- Alle må regne med å måtte forklare detaljer i sin løsning på et kort intervju. (Hvis tiden ikke tillatter det, blir noen innleveringer trukket for et slikt intervju.)

Innføring

Oppgaven går ut på å programmere en parser og Hindley-Milner algoritme, med unifikasjon, for en forenklet del av Haskell. Språket vi betrakter er gitt ved følgende grammatikk:

```
Letter  ::=  a...z          Expr    ::=  Expr1 + Expr | Expr1 - Expr | Expr1
Digit   ::=  0...9          Expr1   ::=  Value * Expr1 | Value : Expr1 | Value
Number  ::=  Digit+         Value   ::=  Number | Ident | ( Expr ) | ...
Ident   ::=  Letter+                     ... |Ident -> Expr | [ Value Value ]
```

Terminalsymbolene er markert med rødt. For lambda bruker vi `|` istendefor Haskells \, mens for å forenkle parsing krever vi paranteser `[...]` rund applikasjon. For eksempel, Haskells '(\a b -> a+b) 2 3', som forkorter '(\a -> \b -> a+b) 2 3', blir `[[|a -> |b -> a+b 2] 3]`. Merk at f.eks. `[4 5]` er et korrekt uttrykk (som ikke er riktig typet).

Våre `Expr` kan ha type `NumT` (f.eks., `5`, eller `[|a->a+1 3]`) eller en funksjonstype (f.eks., `|a->a+2` har type `NumT->NumT`, mens `|a->|b->a+b` har type `NumT->NumT->NumT`). Det er også mulig med polymorfe `Expr`, f.eks., `|x->x` har typen `t0->t0` for en typevariabel `t0`. Du skal implementere typeavledning for lovlige uttrykk uten frie variabler, dvs. `Expr` der hver variabel/`Ident` er introdusert med lambda binding. F.eks., er `x` et lovlig uttrykk, nemlig `Ident` men, som en fri variabel, har det ikke noen bestemt type. I motsetning, er uttrykket `|x->x` lukket (har ingen frie variabler) og har type `t0->t0`.

1 Parsing og ASTer

1. Deklarer `data Ast`, for abstrakte syntakstrær for `Expr`, med konstruktører (med argumenter av passende typer): `App` (for applikasjon), `Add,Sub,Mult,Div` (addisjon, subtraksjon, multiplikasjon og heltallsdivisjon), `Val` (tallverdier/`Number`), `Var` (variabelnavn/`Ident`) og `Lambda` (anonyme funksjonsuttrykk).

2. Definer funksjoner `tokenise::String->[String]` og `parse::String->Ast` som bygger et `Ast` for en korrekt `Expr` input.

2 Typeuttrykk og Unifikasjon

1. Typingsalgoritmen kan kreve underveis et potensielt ubegrenset antall av typevariabler. Vi skal gi dem navn "t1", "t2", "t3",..., så definer en uendelig liste med slike navn (til bruk i del 3). Listekomprehensjon kan gi en konsis definisjon.

2. Definer `data Type` som angir de mulige typeuttrykkene for alle `Expr`. (Én konstruktør er konstanten `NumT`. I tillegg trengs én konstruktør for funksjonstyper og én for typevariabler som brukes alltid underveis av algoritmen og kan forekomme i sluttresultatet.) `Type` skal instansiere klasser `Show` og `Eq`, så vurder om funksjoner `show` eller `(==)` krever en eksplisitt definisjon.

3. Definer funksjon `occursIn::String->Type->Bool` som avgjør om en typevariabel med navnet gitt av strengen i første argumentet forekommer i typeuttrykket i andre argumentet.

4. En likning mellom to typeuttrykk er av `type Eqn = (Type,Type)`. Ved typeavledning unifiserer vi en serie av slike likninger, og henter resultatet unifisert med en typevariabel tilordnet initielt til `Expr` som types. Definer en funksjon `unify::[Eqn]->Type->Type` som unifiserer likninger i første argumentet. Det andre argumentet er (initiell) typevariabel for `Expr` som types. Ved en vellykket unifisering, returnerer funksjonen typeuttrykket unifisert med denne variabelen. Unifikasjon skal skje med occurs-check.

3 Typing

Oppgaven er gitt i punkt **3**, mens punktene **1** og **2** kun er hint som du kan benytte deg av.

1. Hindley-Milner algoritme jobber i en kontekst som gir typer til kjente operatorer samt, etterhvert, typer til typevariabler som har blitt prossesert. En slik kontekst (Γ i forelesningsnotater) er altså en liste med par av kjente og oppsamlede uttrykk og deres typer, som kan fanges av `type Context = [(Ast,Type)]`. (Hint: For (binære) operatorene kan man definere `initialContext::Context` med typer for +, -, * og :. Disse kan representers f.eks. som `Var` "+", `Var` "*", osv., og assosieres med passende `Type`. Definer en slik `initialContext`, dersom du vil bruke den.)

2. Algoritmen genererer deretter en liste med ligninger for et input uttrykk, med en antatt initiell type gitt ved en frisk typevariabel. Definer funksjon `generateEqns` som tar en `Context`, et `Ast` for uttrykket (hvis type skal avledes), initiell typevariabel(navn), samt en (uendelig) liste med ubrukte typevariabel(navn) og returnerer et par med listen av likninger (som skal unifiseres) og gjenstående ubrukte typevariabel(navn). (Hint: Typing av binære operatorene kan eventuelt hardkodes her, hvis man ikke bruker `initialContext`. Hvis man bruker den, må typen til operatoren i det aktuelle `Ast`'et hentes fra riktig `Var`.)

3. Programmer alt som mangler samt hovedfunksjonen `typeOf::String->Type`, som avleder typen til et korrekt `Expr` gitt i argumentstrengen. For eksempel,

`typeOf "x"` gir en feilmelding	`typeOf "|a -> a + [|b -> b 2]"` gir `NumT->NumT`
`typeOf "|x -> x"` gir `t0->t0`	`typeOf "|f -> [f 2]"` gir `(NumT->t1)->t1`
`typeOf "|x -> x+x"` gir `NumT->NumT`	`typeOf "|f -> |a -> [f a]"` gir `(t2->t3)->t2->t3`
`typeOf "[|a -> a+3 4]"` gir `NumT`	`typeOf "|a -> [a a]"` gir Occurs-Check feil

4 Interaksjon med bruker

Funksjoner definert over blir nå brukt i en enkel interaksjon med bruker. Denne starter når brukeren gir kommandoen "`main`" til GHCi. Deretter kan følgende kommandoer gis:

- `t Expr` – typen til uttrykket skrives ut;
- `f navn` – en serie `Expr` leses fra filen `navn`; hvert uttrykk står i en separat linje, uten noen tomme linjer imellom; på skjermen skrives én linje med hvert uttrykk og dets type;
- `q` avslutter interaksjonen og returnerer til GHCi.